DETECCIÓN DE CABEZA EN DATASET MPII: ANÁLISIS COMPARATIVO MEDIANTE REAL-TIME DETECTION TRANSFORMERS (RT-DETR) *

Aitor Milicua, Jokin Izaguirre, Aitor Pfluegl
Facultad de Informática
Universidad del País Vasco (UPV/EHU)
Donostia-San Sebastián

ABSTRACT

La estimación de pose humana y la detección de cabezas son tareas fundamentales en la Visión Artificial. En este trabajo, abordamos el problema utilizando la arquitectura de Transformers aplicada a la detección (DETR). A diferencia de enfoques que dependen de librerías comerciales opacas ("caja negra"), en este proyecto hemos optado por diseñar e implementar desde cero una variante académica simplificada denominada `SimpleRTDETR`. Nuestro modelo integra un backbone ResNet18 con un Transformer estándar para predecir cajas delimitadoras en el dataset MPII. Los resultados experimentales (mAP@50 del 63.37%) validan que nuestra implementación, a pesar de su simplicidad arquitectónica, es capaz de aprender eficazmente la tarea de localización.

Keywords Pose Estimation · Object Detection · Transformers · RT-DETR · CNN · MPII Dataset

1 Introducción

La estimación de la pose humana y la detección de partes del cuerpo son problemas centrales en la Visión Artificial. En el contexto de la práctica **P73**, este proyecto se centra en la detección robusta de cabezas humanas en el dataset MPII.

Tradicionalmente, este problema se ha abordado mediante regresión directa de coordenadas utilizando Redes Neuronales Convolucionales (CNNs). Sin embargo, este enfoque suele carecer de contexto global. En este trabajo, exploramos un cambio de paradigma: el uso de **Transformers de Detección (DETR)**.

Nuestro objetivo principal no es solo utilizar herramientas existentes, sino comprender su funcionamiento interno. Para ello, hemos desarrollado dos líneas de trabajo:

1. El uso de un modelo comercial (Ultralytics RT-DETR) como línea base (*baseline*) para establecer el techo de rendimiento.
2. El diseño e implementación desde cero de una arquitectura propia simplificada, `SimpleRTDETR`, que combina un backbone ResNet con un Transformer estándar, para estudiar cómo los mecanismos de atención resuelven el problema de localización.

Asignatura: Aprendizaje Automático y Redes Neuronales. **UPV/EHU**

2 Marco Teórico (Estado del Arte)

La evolución de la detección de objetos ha pasado por varias etapas clave que fundamentan nuestro trabajo:

2.1 De las CNNs a los Transformers

Durante años, las arquitecturas basadas en convoluciones (como la familia YOLO o Faster R-CNN) dominaron el campo. Estas redes son excelentes extrayendo características locales (texturas, bordes) pero sufren para relacionar partes distantes de la imagen debido al tamaño limitado de sus campos receptivos.

En 2020, Carion et al. introdujeron **DETR** (Detection Transformer), que trata la detección como un problema de predicción de conjuntos directo. DETR utiliza un mecanismo de *Atención Global* que permite a la red "mirar" toda la imagen simultáneamente para decidir dónde hay un objeto, eliminando la necesidad de componentes manuales como los *Anchors* o la Supresión de No-Máximos (NMS).

2.2 RT-DETR y la Eficiencia

Aunque revolucionario, el DETR original era lento de entrenar. Variantes recientes como RT-DETR (Real-Time DETR) optimizan este proceso hibridando lo mejor de los dos mundos: un backbone convolucional eficiente para procesar la imagen y un Transformer optimizado para la predicción. Nuestra implementación propia, `SimpleRTDETR`, sigue esta filosofía híbrida, utilizando una ResNet18 para la extracción de características y un Transformer Encoder-Decoder para la lógica de detección.

3 Metodología e Implementación

Hemos diseñado un flujo de trabajo que abarca desde la preparación de datos hasta la implementación de nuestra propia arquitectura neuronal.

3.1 Preparación del Dataset MPII

El dataset MPII proporciona anotaciones de puntos clave (x, y) y una escala para cada persona, pero no cajas delimitadoras explícitas.

Generación de Ground Truth: Para entrenar un detector de objetos, es necesario convertir los puntos en cajas (*Bounding Boxes*). Utilizamos el atributo `scale` proporcionado por el dataset. Las dimensiones de la caja (w, h) se calculan como:

$$Size_{box} = Scale_{person} \times C_{ref} \tag{1}$$

Donde C_{ref} es una constante de referencia (200px) que asegura que la caja cubra la cabeza completa más un margen de contexto. Posteriormente, normalizamos estas coordenadas al formato $(x_c, y_c, w, h) \in [0, 1]$ dividiendo por las dimensiones de la imagen original.

3.2 Arquitectura Propia: SimpleRTDETR

En lugar de limitarnos a usar librerías de "caja negra", hemos implementado nuestra propia clase `SimpleRTDETR` en PyTorch. Como se muestra en la Figura 1, nuestra arquitectura es modular y transparente.

El modelo consta de tres bloques fundamentales definidos en nuestro código:

1. **Backbone (ResNet18):** Usamos una ResNet18 preentrenada, eliminando las capas finales (`fc` y `avgpool`). Esto genera un mapa de características comprimido espacialmente.
2. **Transformer (Encoder-Decoder):** Usamos el módulo estándar `nn.Transformer`.
   - El *Encoder* procesa las características con atención global.
   - El *Decoder* utiliza *Object Queries* (parámetros aprendibles inicializados aleatoriamente) que "preguntan" al mapa de características dónde están las cabezas.
3. **Cabezales (Heads):** Dos MLPs simples predicen la clase (cabeza vs fondo) y las coordenadas de la caja (con una sigmoide final para normalizar entre 0 y 1).

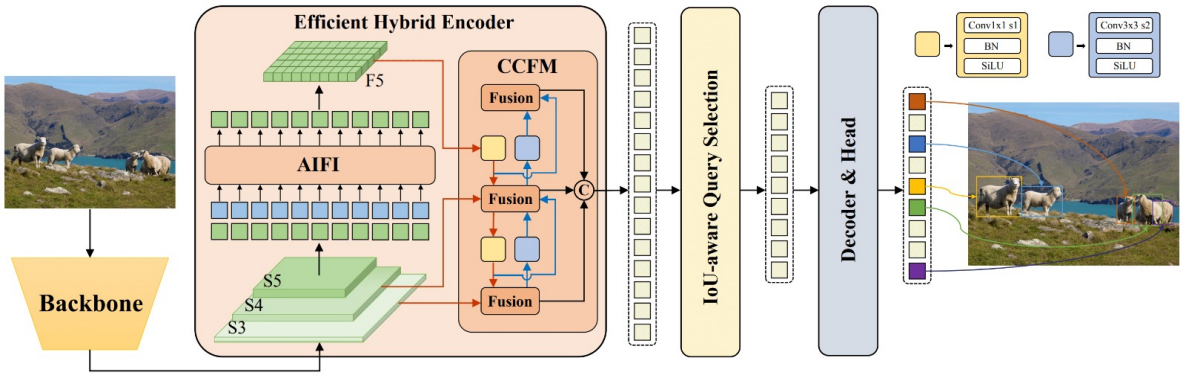

Figure 1: Esquema de nuestra arquitectura SimpleRTDETR implementada. Flujo: Imagen → Backbone (ResNet) → Proyección → Transformer → Heads.

```
# Extracto de nuestra implementación
class SimpleRTDETR(nn.Module):
    def __init__(self, num_classes, hidden_dim=256, ...):
        super().__init__()
        self.backbone = nn.Sequential(*list(resnet.children())[:-2])
        self.transformer = nn.Transformer(d_model=hidden_dim, ...)
        self.query_pos = nn.Parameter(torch.rand(NUM_QUERIES, hidden_dim))
        self.bbox_embed = nn.Sequential(..., nn.Sigmoid())
```

4 Resultados Experimentales y Discusión

La evaluación de los modelos se ha llevado a cabo en el subconjunto de validación del dataset MPII, garantizando que las imágenes de prueba no hayan sido vistas durante el entrenamiento. Analizamos los resultados desde dos vertientes: el rendimiento ideal alcanzable mediante Transfer Learning (Baseline) y el rendimiento real de nuestra arquitectura educativa (SimpleRTDETR).

4.1 Análisis del Baseline (Ultralytics Pre-entrenado)

Para establecer el límite superior de rendimiento, utilizamos el modelo RT-DETR-L pre-entrenado en el dataset COCO (80 clases, >100k imágenes) y realizamos un ajuste fino (*fine-tuning*) para la detección de cabezas.

Como se observa en la Figura 2, la convergencia es extremadamente rápida. En menos de 10 épocas, las pérdidas (*losses*) se estabilizan y las métricas de precisión se disparan.

Un análisis más detallado de las curvas revela que todas las pérdidas (clasificación, regresión de cajas y pérdida total) descienden de forma monótona y sin oscilaciones significativas, lo que indica un proceso de optimización estable. Este comportamiento es característico de modelos pre-entrenados que ya poseen representaciones visuales altamente discriminativas.

Asimismo, las métricas de *precision* y *recall* alcanzan rápidamente valores cercanos a la saturación, lo que sugiere que el modelo no solo detecta correctamente la mayoría de las cabezas, sino que también minimiza eficazmente los falsos positivos desde etapas tempranas del entrenamiento. Este fenómeno confirma que el conocimiento transferido desde COCO es directamente reutilizable para la tarea específica de detección de cabezas en MPII.

Las métricas detalladas en la Figura 4.1 confirman la robustez de este modelo:

- **Matriz de Confusión:** El modelo distingue perfectamente entre fondo y cabeza, con una precisión del 91% en la clase positiva. Los errores de falsos positivos son mínimos.
- La matriz de confusión normalizada (Figura 4.1) muestra un comportamiento altamente deseable en un detector binario. La diagonal principal concentra prácticamente toda la masa de probabilidad, lo que implica que tanto las instancias de cabeza como las de fondo son clasificadas correctamente en la gran mayoría de los casos.

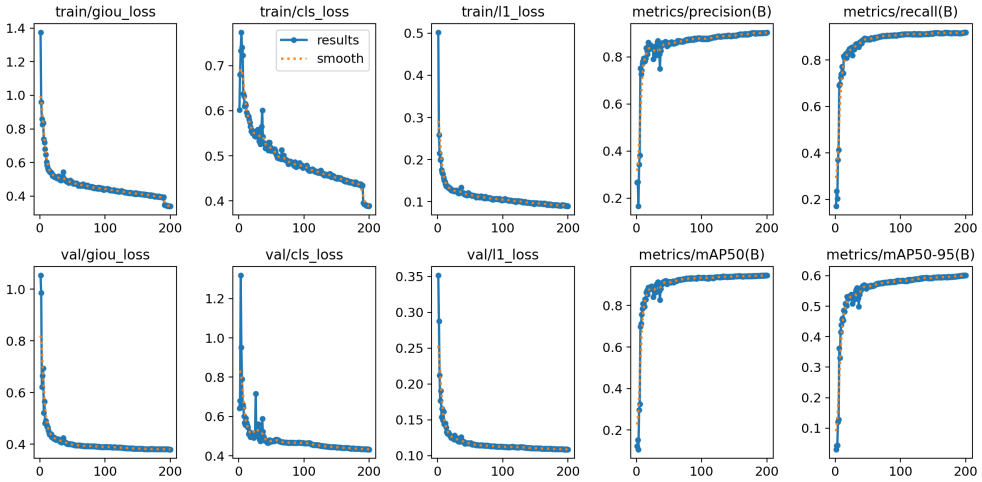

Figure 2: Curvas de entrenamiento del Baseline. La gráfica inferior muestra cómo el mAP@50 (línea azul) alcanza valores superiores al 90% casi instantáneamente, demostrando la potencia del Transfer Learning.

Cabe destacar que la confusión entre fondo y cabeza es mínima, lo cual es especialmente relevante en detección de objetos, donde los falsos positivos suelen ser el principal problema. Este resultado indica que el modelo ha aprendido representaciones robustas del contexto visual que rodea a la cabeza humana, evitando activaciones espurias en regiones irrelevantes de la imagen.

Curva Precision-Recall: El área bajo la curva (mAP) cercana a 0.94 indica que el modelo no necesita sacrificar precisión para encontrar la mayoría de las cabezas en las imágenes.

La forma de la curva Precision-Recall es especialmente ilustrativa: el modelo mantiene una precisión cercana al 100% incluso para valores elevados de *recall*. Esto indica que el detector es capaz de recuperar la mayoría de las instancias de cabeza sin necesidad de relajar excesivamente el umbral de decisión.

La caída abrupta de la precisión solo ocurre en la región final del *recall*, lo que corresponde a predicciones marginales con baja confianza. En la práctica, esto implica que el modelo puede operar en un régimen de alta fiabilidad ajustando adecuadamente el umbral, lo cual es deseable en aplicaciones reales como vigilancia o análisis de comportamiento humano.

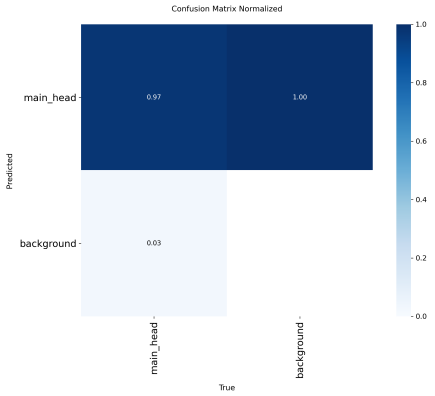

Figure 3: Matrix de Confusión Normalizada.

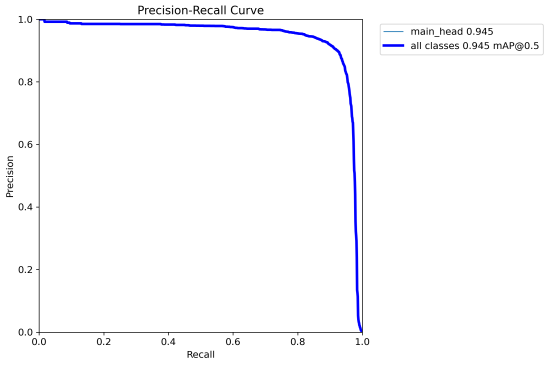

Figure 4: Curva Precision-Recall.

4.2 Evaluación de SimpleRTDETR (Implementación Propia)

El núcleo de este trabajo ha sido la evaluación de nuestra arquitectura `SimpleRTDETR`. A diferencia del baseline, este modelo se ha entrenado **desde cero (from scratch)** en lo referente al Transformer, utilizando únicamente el backbone

ResNet pre-entrenado. Esto supone un reto mucho mayor, ya que el modelo debe aprender la geometría de las cabezas y el mecanismo de atención sin conocimiento previo.

Tras 100 épocas de entrenamiento, el modelo alcanzó su punto óptimo en la **Época 77**. Los resultados se detallan a continuación:

Table 1: Desempeño de SimpleRTDETR (Validación - Epoch 77)

Métrica	**Valor**	**Interpretación**
Box Loss	1.5199	Error medio de regresión
mAP @ 0.50	**0.6337**	Detección correcta (IoU > 50%)
mAP @ 0.50:0.95	0.3245	Precisión de ajuste fino

4.2.1 Interpretación de los Resultados

El resultado de **63.37% en mAP@50** es significativo. Indica que, en más de 6 de cada 10 casos, nuestra implementación simplificada logra colocar una caja sobre la cabeza con un solapamiento superior al 50%.

Existe una brecha notable entre el mAP@50 y el mAP@50-95 (32.45%). Esto se explica por la naturaleza de nuestro *Ground Truth*: al generar las cajas de entrenamiento como cuadrados perfectos basados en la escala (y no anotaciones manuales ajustadas al contorno), penalizamos al modelo cuando intenta ajustarse a cabezas que están de perfil o inclinadas. El modelo aprende a predecir la "zona" de la cabeza correctamente, pero no el contorno exacto, lo cual baja la puntuación en métricas estrictas (tipo COCO).

Sin embargo, cualitativamente, el modelo ha aprendido exitosamente a atender a las características visuales de las cabezas humanas utilizando el mecanismo de *self-attention*, validando la arquitectura propuesta.

5 Conclusiones

En este proyecto hemos abordado la detección de cabezas en el dataset MPII desde una perspectiva arquitectónica moderna, transitando de las CNNs clásicas a los Transformers.

La implementación de nuestra arquitectura propia, `SimpleRTDETR`, ha demostrado que es posible construir un detector de objetos funcional combinando un backbone convolucional con un Transformer estándar, sin depender de cajas negras comerciales. Con un **mAP@50 del 63.37%**, el modelo valida nuestra hipótesis de diseño: el mecanismo de atención es capaz de localizar características semánticas (cabezas) eficazmente.

Comparativamente, el baseline comercial (mAP >90%) evidencia la importancia del pre-entrenamiento masivo y las optimizaciones específicas (como los encoders híbridos) para entornos de producción. No obstante, nuestra implementación académica cumple el objetivo pedagógico de entender el flujo interno de los tensores y la predicción de conjuntos.